

2026 CCF 非专业级软件能力认证

CSP-J 2026 第二轮认证 模拟卷 05-B

入门级

时间：2026 年 8 月 08:30 - 12:00

题目名称	驿路通行证能走多远	连续展区的接收方案	棋盘上的最后一口气	备用线路的严格次优方案
题目类型	传统型	传统型	传统型	传统型
目录	trail	gallery	liberty	backup
可执行文件名	trail	gallery	liberty	backup
输入文件名	trail.in	gallery.in	liberty.in	backup.in
输出文件名	trail.out	gallery.out	liberty.out	backup.out
每个测试点时限	2.0	2.0	2.0	3.0
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
题目分值	100 分	100 分	100 分	100 分
测试点数目	25	25	25	25
测试点是否等分	是	是	是	是
C++ 源程序文件名	trail.cpp	gallery.cpp	liberty.cpp	backup.cpp

本套共 4 题，每题 100 分，每个测试点 4 分，满分 400 分。

提交源程序文件名

对于 C++ 语言	trail.cpp	gallery.cpp	liberty.cpp	backup.cpp
-----------	-----------	-------------	-------------	------------

编译选项

对于 C++ 语言	-O2 -std=c++11 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. main 函数的返回值类型必须为 int，程序正常结束时返回 0。
3. 若无特殊说明，结果采用全文比较，过滤行末空格及文本末尾换行。
4. 选手提交的程序源文件大小不得超过 100 KiB。
5. 程序可使用的栈空间内存限制与题目的内存限制一致。
6. 禁止在源代码中改变编译器参数，例如使用 #pragma 命令。
7. 涉及整数累加、总费用或边权之和时，请自行判断是否需要使用 long long。

驿路通行证能走多远 (trail)

【题目描述】

一条驿路上依次设有 n 个关卡。通过第 i 个关卡会从通行证中扣除 a_i 点额度，所有关卡的扣除量均为正数。

通行证在途中不会补充额度。旅行者通过一个关卡后才能继续尝试下一个关卡；如果某个关卡无法通过，他不能越过它前往更后面的关卡。若旅行者成功走到第 n 个关卡之后，行程也会自然结束。

现有 q 位旅行者。每位旅行者从指定的第 l 个关卡开始，持有 x 点额度，并按编号递增的顺序尝试通过关卡。旅行者不能跳过关卡；若剩余额度不足以通过当前关卡，就立即结束本次行程。

对于每位旅行者，请输出他成功通过的关卡数量，以及结束时通行证中的剩余额度。

旅行者有可能连起点关卡都无法通过，此时通过数量为 0 ，剩余额度仍为最初的 x 。不同旅行者之间互不影响，每次询问都使用一张全新的通行证。

【输入格式】

从文件 `trail.in` 中读入数据。

第一行包含两个整数 n, q 。

第二行包含 n 个正整数 $a_1, a_2, \dots, a_n$ 。

接下来 q 行，每行包含两个整数 l, x ，描述一位旅行者。

【输出格式】

输出到文件 `trail.out` 中。

对于每位旅行者输出一行两个整数，分别表示成功通过的关卡数量和剩余额度。

【样例 1 输入】

```
1 6 4
2 4 2 7 1 3 5
3 1 6
4 2 10
5 4 8
6 6 4
```

【样例 1 输出】

```
1 2 0
2 3 0
3 2 4
4 0 4
```

【样例 1 解释】

第三位旅行者依次通过第 4, 5 个关卡，共使用 $1+3=4$ 点额度。继续通过第 6 个关卡需要 5 点额度，因此他剩余 4 点额度并结束行程。

【样例 2 输入】

```
1 5 3
2 3 3 3 3 3
3 1 14
4 3 6
5 5 3
```

【样例 2 输出】

```
1 4 2
2 2 0
3 1 0
```

【样例 2 解释】

第一位旅行者可以连续通过四个关卡，使用 12 点额度。

【样例 3 输入】

```
1 4 4
2 1 10 2 2
3 2 11
4 1 11
5 3 100
6 4 1
```

【样例 3 输出】

```
1 1 1
2 2 0
3 2 96
4 0 1
```

【样例 3 解释】

第三位旅行者从第 3 个关卡开始，走到驿路末端后仍剩余 96 点额度。

【数据范围】

对于所有测试数据，保证：

- $1 \leq n, q \leq 2 \times 10^5$;
- $1 \leq a_i \leq 10^9$;
- $1 \leq l \leq n$;
- $0 \leq x \leq 10^{18}$ 。

测试点编号	$n \leq$	$q \leq$	特殊性质
1-2	10	10	A、B、C
3-4	100	100	A
5-6	100	100	B
7-9	10^3	10^3	C
10-12	10^4	10^4	A
13-15	3×10^4	3×10^4	B
16-18	5×10^4	10^5	C
19-21	10^5	2×10^5	无
22-25	2×10^5	2×10^5	无

特殊性质 A：所有 a_i 均相等。

特殊性质 B：所有旅行者都从第 1 个关卡开始。

特殊性质 C：每次询问都满足 $x < a_1$ 。

连续展区的接收方案（gallery）

【题目描述】

科技展沿参观路线依次设置了 n 个展区。第 i 个展区使用编号为 a_i 的通信协议。

一台接收器在一次启动期间只能保存至多 k 种不同协议。工作人员希望选择一段连续的展区，使接收器能够覆盖这段路线中的全部协议，并让经过的展区数量尽可能多。

接收器第一次遇到某种协议时会占用一个协议槽，之后再次遇到相同协议不会额外占用槽位。一次启动期间不能清空已经保存的协议再继续参观，因此所选连续区间中出现过的不同协议总数不能超过 k 。

请输出最多能够连续经过多少个展区，以及对应区间的左右端点。若有多个区间长度相同，选择左端点最小的；若左端点也相同，选择右端点最小的。展区编号从 1 开始。

工作人员只启动一次接收器，并且必须选择非空区间。输入顺序就是实际参观顺序，不能交换展区，也不能跳过区间内部的某个展区。

【输入格式】

从文件 `gallery.in` 中读入数据。

第一行包含两个整数 n, k 。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ，表示各展区使用的协议编号。

【输出格式】

输出到文件 `gallery.out` 中。

输出一行三个整数，依次表示最大展区数量、区间左端点和区间右端点。

【样例 1 输入】

```
1 8 2
2 1 2 1 3 4 3 3 2
```

【样例 1 输出】

```
1 4 4 7
```

【样例 1 解释】

第 4 至第 7 个展区使用的协议依次为 3, 4, 3, 3，只包含两种协议，长度为 4。

【样例 2 输入】

```
1 7 1
2 5 5 2 2 2 5 5
```

【样例 2 输出】

```
1 3 3 5
```

【样例 2 解释】

接收器只能保存一种协议，最长的连续相同区间是第 3 至第 5 个展区。

【样例 3 输入】

```
1 6 3
2 8 1 8 2 3 2
```

【样例 3 输出】

```
1 4 1 4
```

【样例 3 解释】

区间 $[1, 4]$ 和 $[3, 6]$ 都包含三种协议且长度均为 4，因此选择左端点更小的 $[1, 4]$ 。

【数据范围】

对于所有测试数据，保证：

- $1 \leq k \leq n \leq 2 \times 10^5$ ；
- $1 \leq a_i \leq 10^9$ 。

测试点编号	$n \leq$	$a_i \leq$	特殊性质
1-2	10	10	A、B、C
3-4	100	100	A
5-6	100	100	B
7-9	10^3	10^3	C
10-12	10^4	10^6	A
13-15	3×10^4	10^9	B
16-18	5×10^4	10^9	C
19-21	10^5	10^9	无
22-25	2×10^5	10^9	无

- 特殊性质 A： $k=1$ 。
- 特殊性质 B： 所有 a_i 均相同。
- 特殊性质 C： 整个序列中不同协议的种数不超过 k 。

棋盘上的最后一口气 (liberty)

【题目描述】

棋盘共有 n 行 m 列，每个交叉点处于以下三种状态之一：

- X: 黑棋;
- O: 白棋;
- .: 空点。

棋子落在棋盘横线与竖线的交叉点上，而不是落在方格内部。判断棋子是否相连时只看上、下、左、右四个方向，斜对角接触的两枚棋子并不相连。

两个同色棋子若在上下或左右方向相邻，就属于同一个棋块。一个白棋块的“气”是与该棋块中任意棋子上下左右相邻的空点，同一个空点只计算一次。

同一个棋块中的所有棋子共同拥有这些气。即使一个空点同时挨着棋块中的多枚棋子，它仍然只是一口气；当一个棋块的最后一口气被占据时，整个棋块会一起被提走，而不是只提走靠近落子位置的棋子。

现在黑方准备在一个空点落下一枚棋子。如果某个白棋块在落子前恰好只有这一口气，那么落子后该白棋块会被提走。多个白棋块可能共用同一个最后的气，因此一次落子可能同时提走多个棋块。

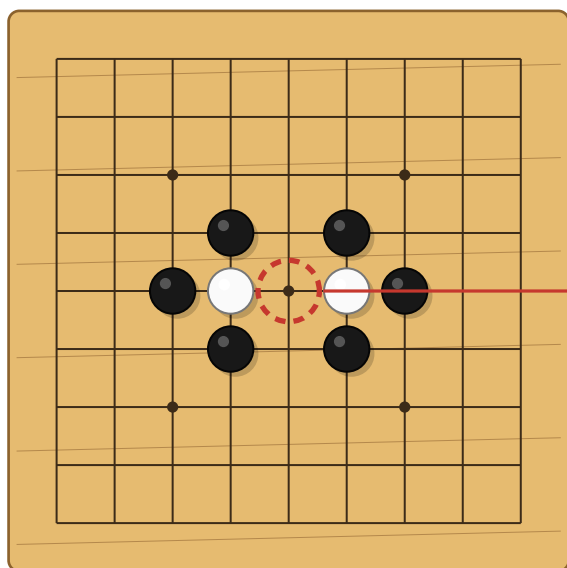
例如，两块互不相连的白棋可能分别被黑棋包围，却恰好都只剩中间同一个空点。黑棋落在这个空点后，两块白棋都会失去最后一口气，提子数应当把两个棋块的棋子数量相加。

请找出能够提走最多白棋的落子位置，并输出提走的白棋数量及该位置。若有多个位置提子数相同，优先选择行号较小的；行号仍相同时，选择列号较小的。

若无论落在哪里都不能提走白棋，输出 0 0

0。本题只统计能够提走的白棋数量，不需要判断黑棋落子后是否形成自杀。

【图示：棋块、气与提子】



图中关系

1. 白棋落在交叉点上。
2. 两枚白棋互不相连，属于两块棋。
3. 每块白棋的另外三侧都被黑棋占据。

红圈空点是两块白棋共用的最后一口气。

黑棋落在红圈处

两块白棋同时失去最后一气，共提走 2 枚。



黑棋



白棋



最后一口气

示意图使用与样例相同的规则。棋盘线的交叉点才是落子位置，红圈并不是棋子，而是可以落子的空点。

【输入格式】

从文件 `liberty.in` 中读入数据。

第一行包含两个整数 n, m 。

接下来 n 行，每行包含一个长度为 m 的字符串，描述棋盘。

【输出格式】

输出到文件 `liberty.out` 中。

输出一行三个整数，依次表示最多能提走的白棋数量、落子行号和落子列号。

【样例 1 输入】

```
1 5 5
2 XXXXX
3 X00.X
4 XX0XX
5 XXXXX
6 XXXXX
```

【样例 1 输出】

```
1 3 2 4
```

【样例 1 解释】

三个白棋连成一个棋块，其唯一的气是第 2 行第 4 列。黑棋落在这里可以提走三枚白棋。

【样例 2 输入】

```
1 3 5
2 XXXXX
3 X0.0X
4 XXXXX
```

【样例 2 输出】

```
1 2 2 3
```

【样例 2 解释】

两枚白棋不属于同一个棋块，但它们唯一的气都是中间的空点，因此可以同时被提走。

【样例 3 输入】

```
1 4 5
2 .....
3 .00..
4 .XX..
5 .....

```


【样例 3 输出】

```
1      0 0 0
```

【样例 3 解释】

白棋块拥有多口气，在任何一个空点落子都不能立即提走它。

【数据范围】

对于所有测试数据，保证：

- $1 \leq n, m \leq 1000$;
- $n \times m \leq 10^6$;
- 棋盘至少有一个空点;
- 每个白棋块在初始棋盘上至少有一口气。

测试点编号	$n, m \leq$	$n \times m \leq$	特殊性质
1-2	6	36	A、B、C
3-4	10	100	A
5-6	20	400	B
7-9	50	2500	C
10-12	100	10^4	A
13-15	300	9×10^4	B
16-18	500	2.5×10^5	C
19-21	1000	5×10^5	无
22-25	1000	10^6	无

- 特殊性质 A：棋盘中没有黑棋。
- 特殊性质 B：每个白棋块都只包含一枚白棋。
- 特殊性质 C：每个白棋块都至少有两口气。

备用线路的严格次优方案（backup）

【题目描述】

通信中心之间共有 n 个节点和 m 条候选双向线路。第 i 条线路连接节点 u_i, v_i ，建设费用为 w_i 。输入保证所有节点能够互相到达，并且所有线路的建设费用两两不同。

如果一套方案中仍存在环路，就一定可以删除环上的一条线路而保持连通，因此题目只考虑恰好使用 $n-1$ 条线路的方案。这样的方案既连通全部节点，又不含环，也就是一棵生成树。

一套完整方案必须选择恰好 $n-1$ 条线路，使所有节点连通。所有完整方案中，费用最小的方案称为最小生成方案，其费用记为 W 。由于边权两两不同，最小生成方案是唯一的。

中心还需要准备一套备用方案。备用方案的总费用必须严格大于 W ，并且在满足这一条件的所有完整方案中费用最小。该费用记为 W_2 。

备用方案只比较线路总费用，不要求与最小生成方案完全不同多少条边。它可以保留最小生成方案中的大部分线路，只替换其中一条或多条线路。由于最小生成方案唯一，任何其他完整方案的总费用都一定严格大于 W 。

请输出 W 和 W_2 。若不存在费用严格大于 W 的完整方案，则令 $W_2=-1$ 。

当原图本身就是一棵树时，所有线路都必须被选择，不存在第二套完整方案，此时按要求输出 -1 。

【输入格式】

从文件 `backup.in` 中读入数据。

第一行包含两个整数 n, m 。

接下来 m 行，每行包含三个整数 u_i, v_i, w_i ，表示一条候选线路。

【输出格式】

输出到文件 `backup.out` 中。

输出一行两个整数，分别表示最小生成方案费用 W 和严格次优方案费用 W_2 。

【样例 1 输入】

```
1 4 5
2 1 2 1
3 2 3 2
4 3 4 3
5 1 4 6
6 1 3 4
```

【样例 1 输出】

```
1 6 8
```

【样例 1 解释】

最小生成方案选择费用为 1, 2, 3 的三条线路，总费用为 6。用费用为 4 的线路连接节点 1, 3，并替换原方案中费用为 2 的线路，可以得到费用为 8 的严格次优方案。

【样例 2 输入】

```
1 4 3
2 1 2 1
3 2 3 2
4 3 4 3
```

【样例 2 输出】

```
1 6 -1
```

【样例 2 解释】

三条线路组成一棵树，任何一条都不能删除，因此不存在备用方案。

【样例 3 输入】

```
1 4 6
2 1 2 1
3 2 3 4
4 3 4 6
5 1 3 5
6 2 4 7
7 1 4 10
```

【样例 3 输出】

```
1 11 12
```

【样例 3 解释】

最小生成方案选择费用为 1, 4, 6 的线路，总费用为 11。加入费用为 5 的线路并删除原方案中费用为 4 的线路，可以得到费用为 12 的备用方案。

【数据范围】

对于所有测试数据，保证：

- $2 \leq n \leq 2000$;
- $n-1 \leq m \leq 10000$;
- $1 \leq u_i, v_i \leq n$, 且 $u_i \neq v_i$;
- $1 \leq w_i \leq 10^9$;
- 输入的无向图连通;
- 任意两个节点之间至多有一条线路;
- 所有线路的建设费用两两不同。

测试点编号	$n \leq$	$m \leq$	特殊性质
1-2	8	15	A、B、C
3-4	50	100	A
5-6	100	500	B、C
7-9	200	10^3	C
10-12	500	2×10^3	A
13-15	800	4×10^3	B
16-18	10^3	5×10^3	C
19-21	1500	8×10^3	无
22-25	2000	10^4	无

特殊性质 A： $m=n-1$ 。

特殊性质 B： 输入中的线路已经按照费用从小到大排列。

特殊性质 C： $n \leq 100$ 。